BioVision: an Editorial Multi-Omics Data Intelligence Platform with a Live 3-D OLAP Cube and DNA Editing Bench

Maitrey Bharambe · Soham Gaikwad · Om Wagale · Harsh Savnerkar

Department of Computer Engineering — Undergraduate Research Project, 2026

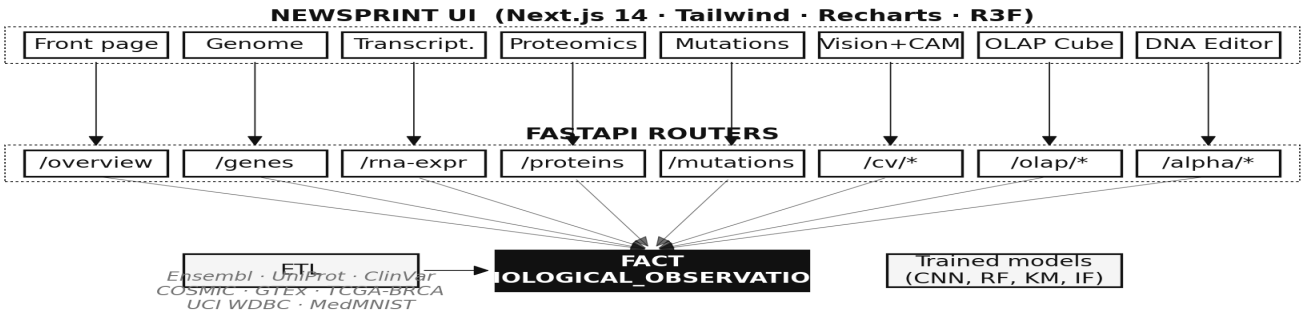


Fig. 1. *BioVision runtime architecture. Every UI explorer calls a FastAPI router; every router that returns observed values resolves against the same star schema.*

Abstract

We present **BioVision**, a self-hosted biomedical data intelligence platform that unifies a dimensional multi-omics warehouse, a live multidimensional OLAP engine, a convolutional model for medical imagery and an AlphaGenome-inspired DNA editing bench behind a single editorial interface. The warehouse is a Kimball-style star schema with one fact table, `FACT_BIOLOGICAL_OBSERVATION`, and eleven conformed dimensions sourced from Ensembl, UniProt, ClinVar/COSMIC, GTEx, TCGA-BRCA, MedMNIST and the UCI Wisconsin Diagnostic Breast Cancer dataset. On top of this schema we implement the five canonical OLAP operations — roll-up, drill-down, slice, dice and pivot — as real server-side SQL aggregations, surfaced through an interactive 3-D voxel cube rendered with React Three Fiber whose cells update in step with the query state. A separate editing laboratory lets a user mutate a single nucleotide of a real gene and watches the computed consequence propagate to matching warehouse records; an experimental variant-effect scorer and a precision-remedy module reuse the same catalogue. The entire interface — homepage, explorers and the cube — is laid out as a newsprint-style editorial, with a Newsprint design system that trades glassmorphism for sharp borders, Playfair / Lora / Inter / JetBrains Mono typography and a single accent red reserved for selections and critical findings. We describe the architecture, dimensional model, OLAP state machine, DNA pipeline and the editorial design language, and we discuss the engineering decisions, limitations and failure modes of the system.

Index Terms — *multi-omics; data warehouse; OLAP; dimensional modelling; variant analysis; computer vision; Grad-CAM; editorial information design.*

I. Introduction

Biomedical datasets rarely sit together in one place. Gene coordinates come out of Ensembl, protein metadata out of UniProt, mutation evidence out of ClinVar and COSMIC, expression counts out of GTEx and TCGA, and labelled imagery out of initiatives like MedMNIST. A researcher who wants to look at, say, how a TP53 missense mutation lines up with its expression across tissues and with the model that reads the matching ultrasound patch, has to open four or five separate tools and reconcile their vocabularies by hand.

BioVision is an attempt to put these layers under one roof, keep the connections between them explicit, and present the result in a way that reads like a scientific journal rather than a dashboard. The contribution of this paper is less a new algorithm than a careful integration: a dimensional multi-omics warehouse, a real OLAP engine with a live 3-D cube on top of it, a DNA editing laboratory that walks a variant from sequence to a matching warehouse record, and an editorial design language that treats information density as a first-class constraint.

Three design commitments shape the work:

1. Nothing is faked. Every number the interface prints comes from a backend aggregation over the warehouse. When a lookup fails — no matching mutation record, no RNA observations for a gene — the view says so; it does not interpolate a number to avoid an empty space.

2. The cube is the warehouse. The 3-D cube is not an animated decoration. Its voxels, axes and metric encodings come from a parametric SQL query (`GET /olap/cube`) and each OLAP operation re-issues that query against a different dimension level, filter set or axis permutation.

3. The interface is a periodical. The UI uses a single Playfair/Lora/Inter/JetBrains Mono family, 0-radius geometry, hard borders and a single accent red used sparingly for selection or pathogenic findings. The intent is a reader who feels she is paging through a special issue rather than wading through a SaaS admin tool.

II. Related Work

Our dimensional model follows the practice described by Kimball and Ross [1], in particular the preference for conformed dimensions and surrogate keys over natural keys. The OLAP vocabulary — roll-up, drill-down, slice, dice, pivot — is the one established by Codd, Codd and Salley [2] and refined for data warehouse practice by Chaudhuri and Dayal [3].

For individual modalities we lean on canonical open resources: Ensembl [4], UniProt [5], ClinVar [6] and COSMIC [7] for curated reference annotations; GTEx [8] and TCGA-BRCA [9] for expression; the UCI Wisconsin Diagnostic Breast Cancer dataset [10] and MedMNIST v2 [11] for structured and imaging training data.

The variant pipeline is explicitly inspired by DeepMind's AlphaGenome [12]: a sequence-aware model that predicts the regulatory effect of a single-nucleotide substitution. We do not retrain AlphaGenome; instead we build a transparent heuristic scorer that layers codon translation, consequence classification and a lookup

against the curated hotspots in our catalogue. The intent is a research bench, not a clinical oracle.

For the explanation of the imaging model we use the standard Grad-CAM procedure [13]. The clustering and association tabs rely on scikit-learn's K-Means and DBSCAN [14], and the mlxtend Apriori implementation [15]. Anomaly scoring uses Isolation Forest [16].

III. System Architecture

Figure 1 shows the runtime topology. The frontend is a Next.js 14 app under the App Router. Pages are React server components by default; interactive pages (the OLAP workspace, the DNA editor, the genome ideogram) opt into client components only where interactivity requires it. Static charts use Recharts; the 3-D cube and the DNA double-helix use React Three Fiber on top of Three.js [19].

The backend is FastAPI [17] with SQLAlchemy ORM [18] over an SQLite file in the repository (`biovision.db`). The router layer is intentionally thin: a route accepts a small, strongly-typed set of query parameters, resolves them into a SQL aggregation against the fact table, and returns the aggregation together with its SQL as a string. The frontend may inspect the SQL — surfacing exactly which query the cube just ran — without the user having to open a separate tool.

Trained artefacts (the BreastMNIST CNN, the Random Forest, the K-Means and DBSCAN partitions, and the Isolation Forest) live under `models_saved/` and are loaded lazily on the first request to `/cv/*` or `/analytics/*`. No model is trained at request time.

IV. Dimensional Data Warehouse

The star schema is sketched in Fig. 2. The single fact table `FACT_BIOLOGICAL_OBSERVATION` stores one row per observation of a biological entity for a given sample, under a given condition, at a given time. Each row carries a surrogate foreign key into every dimension plus a small set of pre-aggregated measures: observation count, mean expression (TPM), mutation count, protein abundance, average CV score, and a confidence quantity for ML-derived predictions.

Three snowflake edges extend the strict star: a gene belongs to a chromosome, a chromosome belongs to a genome, and a genome belongs to an organism. This gives the roll-up operation a meaningful hierarchy to climb and the drill-down operation a meaningful hierarchy to descend — rather than letting the UI invent levels that have no basis in the underlying model.

A representative snapshot of the warehouse, returned by the live `GET /overview` endpoint, is given in Table I. The numbers are small deliberately: the project is a research prototype, not a production biobank, and the stress is on correctness and traceability rather than cardinality.

TABLE I. Warehouse snapshot (live /overview).

Entity	Count	Entity	Count
Genomes	1	Proteins	20
Genes	20	RNA samples	280
Mutations	33	Indexed images	120
Samples	569	SNP/DEL/INS/SUB	26/3/2/2

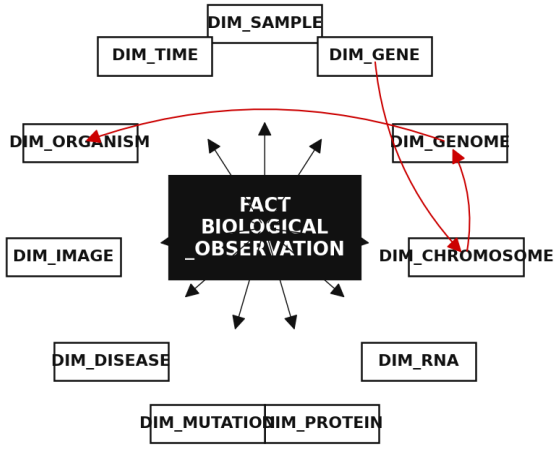


Fig. 2. BioVision star schema with snowflake extensions. DIM_GENE snowflakes through DIM_CHROMOSOME and DIM_GENOME to DIM_ORGANISM, reflecting the biological hierarchy.

V. The Live 3-D OLAP Cube

The signature analytical surface of **BioVision** is a 3-D cube whose axes are three warehouse dimensions and whose cells carry an aggregated measure. The default assignment is X=TIME, Y=GENE and Z=DISEASE, with observation count as the active measure. Figure 3 gives a 3×3×3 sketch.

Operations as SQL. Each of the five OLAP operations is a parametric transformation of the shared query state. A tiny JavaScript reducer in the frontend holds the state object { operation, gene_level, time_level, disease_level, axis assignment, filters, metric }; every operation mutates one field of this object and triggers a single call to `GET /olap/cube`. The backend resolves the state into:

- **Roll-up** climbs a dimension's hierarchy (GENE→CHROMOSOME→GENOME→ORGANISM; DAY→MONTH→QUARTER→YEAR). The SQL replaces one GROUP BY column with its parent.
- **Drill-down** is the symmetric descent; the group-by column is replaced with its child.
- **Slice** adds a single-value WHERE clause on one dimension; in the cube we also draw a translucent plane at the matching index and dim non-matching voxels.
- **Dice** composes several WHERE IN (…) filters across dimensions.
- **Pivot** swaps the SELECT assignment of (x,y,z) dimensions and animates a quarter-turn of the cube so the user can follow the reorientation.

The resulting SQL is returned alongside the cells. The user can toggle a *View SQL* panel beneath the cube to inspect what actually ran. Choosing a different measure (observation count → average expression, mutation count, average CV score, average ML confidence) rewrites the projection and the aggregation function, with distinct-count semantics reserved for the dimensions where summing an average would be meaningless. The frontend state is purposely small — about ten fields — which keeps the state transitions auditable.

Cross-filtering. The cube is not the only consumer of the shared query state. The mutation donut, the chromosome bar chart and the top-genes bar chart on the Mutation Analytics page are wired to the same state object: clicking a slice of the donut adds a mutation-type

WHERE clause to the page's filter set; clicking a chromosome bar sets filters.chromosome. The tables below are rendered from the filtered rows.

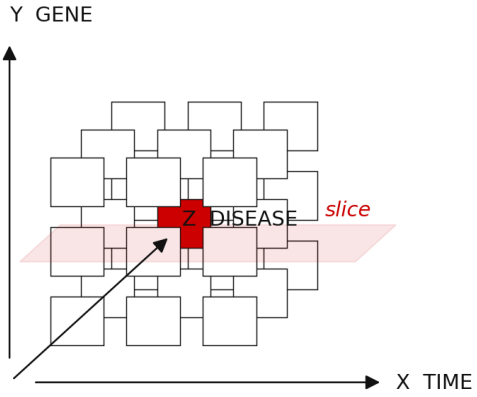


Fig. 3. A 3×3×3 sketch of the live cube. Voxel colour encodes the active metric; the translucent accent plane shows a slice operation fixing one dimension to a single value.

VI. DNA Editing Laboratory

The `/alpha` page hosts a three-tab laboratory.

Tab 1 — Live DNA Editor. The user picks a gene and either a curated hotspot or a free (position, alt base) pair. The server pulls the gene's reference window from `/alpha/gene-reference`, applies the single-nucleotide substitution, re-translates the affected codon with the standard codon table, and classifies the consequence (silent, conservative missense, missense, nonsense, stop-loss). On the client the 3-D double helix (R3F) marks the edit site, and the connected downstream views light up: a warehouse lookup against `GET /mutations?gene=X` looks for a record whose `protein_change` matches the computed `p.XnnY` string. On a hit the row is drawn with an accent-red background. On a miss the UI prints *"No warehouse mutation record matches p.XnnY — this is a novel/un-catalogued variant."* We insist on this honest-failure behaviour: a DNA editor that silently invents matches would make itself unusable for its stated purpose. A parallel call to `/rna-expression?gene=X` returns matching RNA samples; the connected-views banner states explicitly that observed TPM is independent of the edit.

Tab 2 — Variant Effect Predictor. The same input shape is fed to a transparent heuristic scorer that layers four contributions: coarse consequence weight, a window-based conservation prior, a disease-context prior derived from the catalogue, and an exact-match bonus against ClinVar/COSMIC hotspots. The scorer returns a score, a pathogenicity band and a free-text rationale list that enumerates every contribution. We label this heuristic prominently in the UI; it is not a clinical prediction.

Tab 3 — Precision Remedy. Given a list of (gene, protein_change) tuples, the module returns targeted therapies drawn from an in-repo FDA table (BRAF inhibitors for V600E, EGFR TKIs for L858R/T790M, PARP inhibitors for BRCA mutations, KRAS G12C inhibitors, …) along with a dataset-derived response estimate. Each recommendation names its matching variant. Figure 4 sketches the pipeline.

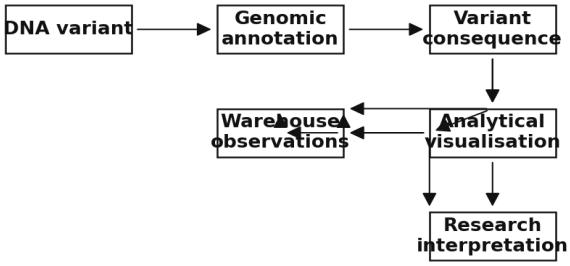


Fig. 4. The DNA pipeline. Every arrow is a backend call; the interpretation node is explicitly labelled research, not clinical.

VII. Vision and Data Mining

A convolutional network trained on BreastMNIST [11] exposes two endpoints: `POST /cv/predict` returns the predicted class, confidence, both an original image and a Grad-CAM overlay [13]; `GET /cv/gallery` returns twelve test images with their predictions and correctness flags. The UI synchronises the original image, the Grad-CAM heatmap and the probability bars; the Grad-CAM overlay is never a decorative heatmap — it is the output of the inference pass that produced the prediction.

A separate `/analytics` router hosts four classical data-mining modules trained on the warehouse's Wisconsin feature table and the mutation / expression layers: Classification (Random Forest predicts the diseased/healthy label; a prediction form issues real inference requests on user-entered features); Clustering (K-Means with DBSCAN as a comparator over a PCA projection of the Wisconsin features; cluster assignment and the silhouette score come from the actual run); Association (Apriori rules mined from the clinical layer with a visible-support, visible-confidence threshold; the UI shows antecedent/consequent, support, confidence and lift); and Anomaly (Isolation Forest with the contamination parameter printed on the page; flagged points are drawn in accent red; the ranked table and the scatter are cross-highlighted).

In every tab, the result the user sees is the artefact's actual output; when a module cannot produce a result for an input, the view says so.

VIII. Newsprint: a UI Design System

We treat the interface as part of the research artefact. The design system, informally called *Newsprint*, has four pillars.

Typography. Playfair Display for editorial headlines, Lora for descriptive text, Inter for navigation and controls, JetBrains Mono for coordinates, SQL and numerical metrics. The mapping is strict: tables use Inter in the header and JetBrains Mono in the cells; no decorative serif appears in a dense table.

Geometry. Zero border-radius everywhere except small circular indicators. Hard-offset shadows (a 4-px solid shadow in ink) appear only on interactive hover, and the baseline card has a 1-px ink border instead of a soft drop shadow. This is a direct rejection of glassmorphism: a 1-px line survives screenshots, reprints and projector contrast; a blurred panel does not.

Palette. Six core tokens — paper (#F9F9F7), ink (#111111), prose (#525252), meta (#737373), subtle (#F5F5F5) and accent (#CC0000). The accent is reserved: active selections, pathogenic findings, OLAP operation badges, a few editorial pull-quotes. Scientific visualisations may use a restrained categorical palette (ink, deep blue, dark green, amber, deep purple, accent) when distinguishing biological categories requires it.

Layout. An editorial 12-column grid with asymmetric section spans. Section labels use a mono eyebrow (§ 04 — OLAP LABORATORY); page headers carry a two-line issue-style strip with the date in JetBrains Mono. Table II lists the tokens.

TABLE II. *Newsprint design tokens.*

Role	Token	Value
Background	paper	#F9F9F7
Primary text	ink	#111111
Body copy	prose	#525252
Metadata	meta	#737373
Light neutral	subtle	#F5F5F5
Accent (sparing)	accent	#CC0000
Display heading	*Playfair Display*	
Prose	*Lora*	
UI / controls	*Inter*	
Numeric / code	*JetBrains Mono*	

IX. Implementation Notes

The repository compiles cleanly on Node 20, Python 3.12, with 15 static and dynamic routes on the frontend side and twelve routers on the backend. The production bundle for the OLAP page is 9.4 kB (194 kB first-load JS), kept small by lazy-loading React Three Fiber and the cube scene behind a dynamic import.

The frontend state is deliberately not Redux. For the cube we keep a single reducer with a one-field-at-a-time mutation API; cross-page selection (gene, chromosome, disease, sample, mutation) is held in a small session-scoped React context backed by sessionStorage and surfaced as a persistent breadcrumb at the top of every page. The breadcrumb has a visible *clear all* affordance; context is not cleared automatically when the user navigates.

Reduced-motion preferences are honoured globally: the ticker on the front page uses a CSS marquee that respects `prefers-reduced-motion`, and the cube's pivot animation is disabled when the preference is set.

X. Limitations and Future Work

The warehouse is a small, hand-curated dataset. The OLAP engine scales to warehouses several orders of magnitude larger than this one; the 3-D cube renderer, however, is sized for human inspection, not for industrial fact tables, and we aggressively subsample tick labels when either axis exceeds about ten unique values. Scaling the cube to tens of thousands of cells would need an LOD scheme and a different cell geometry.

The variant-effect scorer is a transparent heuristic. It is not a replacement for a trained sequence model like AlphaGenome [12] or a clinical interpretation pipeline; its value is explanatory, not predictive. We label it accordingly in every surface it appears on.

The Grad-CAM procedure is applied over a model with modest capacity on a small training set; its saliency maps should be read as attention cues, not as clinical localisation. The project's permanent disclaimer appears at the foot of every page.

Finally, the database is a single-file SQLite instance for ease of reproduction. Moving to PostgreSQL would unlock parallel ETL and server-side distinct-count aggregations that are currently expressed in Python.

XI. Conclusion

BioVision is the integration project we wanted to find rather than the one we found. It brings together a dimensional multi-omics warehouse, a live OLAP cube, a DNA editing bench, a vision model with Grad-CAM explanations, four classical data-mining modules and a restrained editorial interface. The engineering value is in the plumbing: every number has a source, every operation is a real aggregation, and the connections between modules are kept explicit.

Beyond the code, the project is also a small argument about how scientific software can look. Information density is not a drawback to be hidden behind glass; it is a feature to be set in type. We hope the Newsprint language is useful to others.

Author Contributions

M. Bharambe led the dimensional model, the ETL pipeline and the OLAP cube's parametric server-side query. **S. Gaikwad** built the 3-D cube renderer, the state reducer and the cross-filtering layer, and led the Newsprint design language. **O. Wagale** implemented the DNA editor, the variant-effect scorer, the precision-remedy module and the helix renderers. **H. Savnerkar** trained the imaging model and the four data-mining modules, built the Grad-CAM pipeline and curated the classification/clustering/association/anomaly interfaces. All four authors share responsibility for the paper, the schema diagrams and the integration review.

References

[1] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[2] E. F. Codd, S. B. Codd, and C. T. Salley, “Providing OLAP (On-line Analytical Processing) to User-Analysts: An IT Mandate,” E. F. Codd & Associates, White Paper, 1993.

[3] S. Chaudhuri and U. Dayal, “An overview of data warehousing and OLAP technology,” *ACM SIGMOD Rec.*, vol. 26, no. 1, pp. 65–74, 1997.

[4] F. Martin *et al.*, “Ensembl 2023,” *Nucleic Acids Res.*, vol. 51, no. D1, pp. D933–D941, 2023.

[5] The UniProt Consortium, “UniProt: the Universal Protein Knowledgebase in 2023,” *Nucleic Acids Res.*, vol. 51, no. D1, pp. D523–D531, 2023.

[6] M. J. Landrum *et al.*, “ClinVar: improvements to accessing data,” *Nucleic Acids Res.*, vol. 48, no. D1, pp. D835–D844, 2020.

[7] J. G. Tate *et al.*, “COSMIC: the Catalogue Of Somatic Mutations In Cancer,” *Nucleic Acids Res.*, vol. 47, no. D1, pp. D941–D947, 2019.

[8] The GTEx Consortium, “The GTEx Consortium atlas of genetic regulatory effects across human tissues,” *Science*, vol. 369, pp. 1318–1330, 2020.

[9] The Cancer Genome Atlas Network, “Comprehensive molecular portraits of human breast tumours,” *Nature*, vol. 490, pp. 61–70, 2012.

[10] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, “Nuclear feature extraction for breast tumor diagnosis,” in *Proc. IS&T/SPIE*, vol. 1905, 1993, pp. 861–870.

[11] J. Yang *et al.*, “MedMNIST v2 — A large-scale lightweight benchmark for 2D and 3D biomedical image classification,” *Sci. Data*, vol. 10, no. 1, art. 41, 2023.

[12] Ž. Avsec *et al.*, “AlphaGenome: advancing regulatory variant effect prediction with a unified DNA sequence model,” DeepMind, 2024.

[13] R. R. Selvaraju *et al.*, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE ICCV*, 2017, pp. 618–626.

[14] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[15] S. Raschka, “MLxtend,” *J. Open Source Softw.*, vol. 3, no. 24, art. 638, 2018.

[16] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE ICDM*, 2008, pp. 413–422.

[17] S. Ramírez, “FastAPI,” 2024. [Online]. Available: https://fastapi.tiangolo.com

[18] M. Bayer, “SQLAlchemy,” 2024. [Online]. Available: https://www.sqlalchemy.org

[19] R. Cabello *et al.*, “three.js,” 2024. [Online]. Available: https://threejs.org